

TECHNICAL ARCHITECTURE & ENGINEERING REPORT

End-to-End IoT Weather Station and Multi-Step Temperature Forecasting System

Downtown District (D01) | 6-Hour Horizon | Edge-Oriented Architecture

By: Mohamed Mostafa Nasr
Supervised by: Eng. Mahmoud Mohamed

System Item	Value	Engineering Interpretation
Historical dataset	526,080 records	Notebook quality check: 0 duplicates / 0 missing values
Modeling population	26,298 Downtown rows	Single-district model for Downtown (D01)
Final feature space	18 features	Strict feature order stored with the model artifact
Forecast output	6 horizons	t+1 through t+6, direct multi-output prediction
Mean test MAE	1.79 °C	Across six horizons
Mean test RMSE	2.27 °C	Across six horizons
Mean test R ²	0.9422	Across six horizons

Report basis

This report synthesizes the supplied Proteus schematic, embedded firmware, Python logger screenshot, captured CSV sample, documented ML notebook, inference.py implementation, and model-evaluation output. Where the artifacts disagree or leave an interface unspecified, the discrepancy is identified explicitly rather than silently normalized.

Contents

1. Executive Summary & System Overview
2. Hardware Architecture & Circuit Design (Proteus)
3. Embedded Firmware Architecture (C++)
4. Simulation Interface & Telemetry Logger (Python Middleware)
5. Machine Learning Pipeline & Mathematics
6. Model Training & Evaluation Breakdown
7. Real-Time Hardware Inference & Production Deployment
8. Verification, Results & Failure Modes
9. Engineering Conclusions & Recommended Integration Path
10. Appendix A. End-to-End Interface Contract
11. Appendix B. Model Feature Dictionary
12. Appendix C. Artifact Traceability & Technical References

Critical architecture observation

The sensing and logging path is currently narrower than the ML feature contract. The Arduino/Proteus subsystem supplies temperature, humidity and light intensity, while the trained model requires 18 features including rainfall, wind speed, cloud cover, weather-condition indicators, cyclical time features, and event flags. A production deployment therefore needs an explicit aggregation/enrichment layer or additional sensors before the current model can be considered fully closed-loop.

1. Executive Summary & System Overview

1.1 High-Level Purpose

The project implements an end-to-end weather sensing and forecasting workflow that begins with a simulated embedded weather station and terminates in a six-hour temperature forecast for the Downtown district (D01). The embedded layer demonstrates real-time acquisition of temperature, relative humidity and ambient light through an Arduino UNO, while the machine-learning layer operates on a richer hourly meteorological dataset and predicts six future temperatures simultaneously.

The architecture is intentionally split into a sensing/telemetry plane and a forecasting plane. The sensing plane is deterministic and lightweight: a DHT11 provides digital temperature/humidity measurements, an LDR is read through an ADC channel, and UART exposes the telemetry stream. The forecasting plane performs temporal feature construction, direct multi-horizon target alignment, chronological validation, multi-output linear regression, and joblib serialization.

1.2 End-to-End System Architecture

```
TRAINING: [weather_processed.csv]
-> [Profiling + Validation]
-> [Downtown Filter + Feature Engineering]
-> [Targets t+1 ... t+6]
-> [Chronological 80/20 Split]
-> [Multi-Output Linear Regression]
-> [temperature_linear_model.pkl]

RUNTIME: [Proteus Sensors]
-> [Arduino UNO / C++]
-> [UART 9600 / Virtual COM]
-> [Python Logger]
-> [Hourly Aggregation + Feature Adapter]
-> [Strict 18-Feature Schema]
-> [Linear Regression]
-> [Forecast t+1 ... t+6]
```

The figure above is intentionally expressed as a traceable block architecture rather than an abstract ML diagram. It distinguishes the model-training data source from the embedded telemetry source and highlights the missing integration step required to make the runtime path schema-compatible.

1.3 System Boundaries and Current Implementation Status

Function	Evidence from supplied artifacts	Implementation status
Temperature + humidity sensing	DHT11 in Proteus + Arduino firmware	Implemented
Light sensing	LDR + 10 kΩ divider + A0	Implemented
UART telemetry	Arduino Serial at 9600 baud	Implemented
CSV persistence	Python logger opens sensor_dataset.csv in append mode	Implemented in supplied logger view

Function	Evidence from supplied artifacts	Implementation status
Historical weather modeling	526,080 x 14 input dataset; 26,298 Downtown rows used	Implemented
6-hour forecasting	Six direct target columns + multi-output LinearRegression	Implemented
Runtime feature adaptation	inference.py builds and orders the model feature vector	Implemented as software demonstration
Rain/wind/cloud sensing on current hardware	No corresponding sensors visible in Proteus schematic	Not implemented
2-second to hourly aggregation	Not shown in supplied artifacts	Integration required
Automatic serial reconnect	Visible logger catches open-port errors and exits	Not implemented
Full production status propagation	Firmware emits a diagnostic string, inference expects sensor_status	Interface gap

2. Hardware Architecture & Circuit Design (Proteus)

2.1 Schematic Walkthrough

The Proteus schematic centers on an Arduino UNO (UNO1) connected to two sensor channels and a serial visualization path. U1 is a DHT11 module. Its data line is routed to the Arduino digital pin configured as DHTPIN=2, while the supply and ground are tied to the corresponding power rails. The LDR channel is implemented as a voltage divider composed of the photoresistor and a 10 kΩ resistor, with the divider node routed to analog input A0.

The serial path is connected to the UNO UART pins and terminates at a Proteus Virtual Terminal for immediate observation of telemetry. A COMPIIM block is also present in the schematic, providing a conventional host-serial interface location; however, the captured run visibly demonstrates data at the Virtual Terminal, so COMPIIM should not be treated as the active endpoint without further verification.

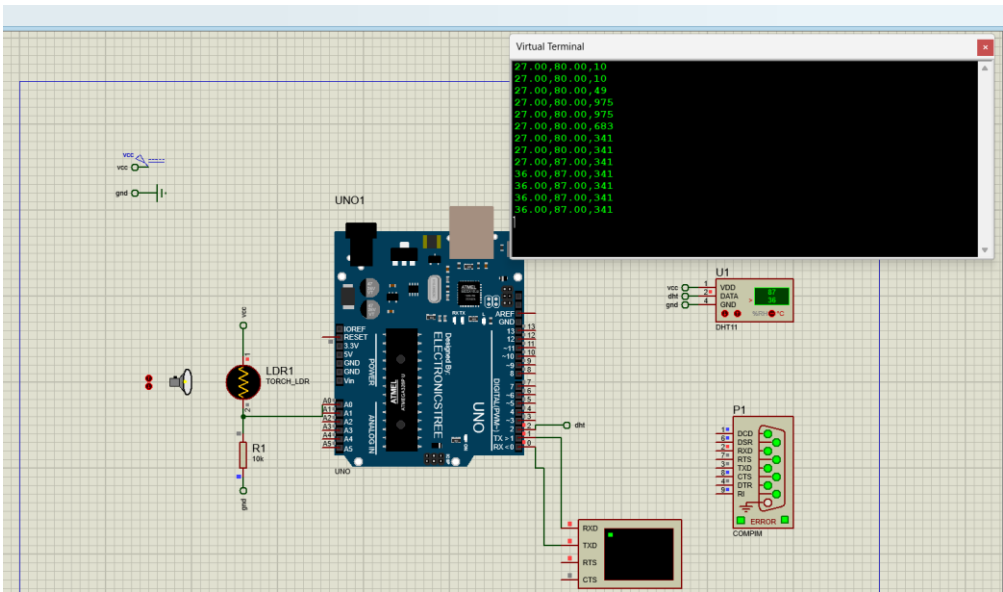


Figure 1. Supplied Proteus schematic and live Virtual Terminal output.

2.2 Interfacing and Signal Map

Component	Arduino pin	Interface / electrical function	Evidence / behavior
DHT11	D2	Digital single-wire sensor interface	Temperature + humidity sampled by firmware
LDR	A0	Analog voltage divider / ADC input	Brightness represented as `analogRead(A0)` value
UART TX	D1 / TX	Asynchronous serial output	Telemetry transmitted at 9600 baud
UART RX	D0 / RX	Asynchronous serial input	Connected in the serial test path; no input parsing in firmware
Virtual Terminal	UART pair	Simulation-side serial monitor	Displays comma-delimited sensor values
R1	10 kΩ to GND	LDR divider reference	Converts light-dependent resistance into

Component	Arduino pin	Interface / electrical function	Evidence / behavior
			ADC voltage

2.3 Circuit Operating Modes

Mode	Trigger	Observed behavior	Engineering implication
Normal acquisition	DHT11 returns finite temperature and humidity	Firmware transmits <code>`Temperature,Humidity,LightIntensity`</code>	Nominal telemetry path
DHT sensor fault	<code>`isnan()`</code> evaluates true	Firmware emits diagnostic text plus light reading	Not machine-readable as a structured status packet
Serial transport failure	COM port unavailable or occupied	Python logger catches open-port exception and exits	Requires operator restart / retry logic
Inference sensor fallback	<code>`sensor_status != Normal`</code> in inference layer	Last valid weather values are reused	Protects inference from transient missing values

2.4 Simulation Boundary Conditions

- The Proteus circuit directly demonstrates temperature, humidity and light acquisition; it does not show dedicated rain, wind or cloud-cover sensors.
- The machine-learning feature space therefore cannot be generated solely by the present hardware without an additional weather-data source or additional sensors.
- The embedded firmware samples every 2 seconds, whereas the ML training data is hourly. A production bridge must define an explicit temporal aggregation policy.
- The simulation validates signal flow and software interoperability, but it does not by itself validate outdoor sensor calibration, enclosure effects, RF reliability, or long-term drift.

3. Embedded Firmware Architecture (C++)

3.1 Core Execution Logic

The supplied Arduino firmware is deliberately compact. Initialization occurs in ``setup()``, where UART is configured for 9600 baud and the DHT11 driver is started. The ``loop()`` function then performs three acquisition operations in a fixed sequence: read humidity, read temperature, and read the LDR through the ADC. A validity check determines whether a structured packet or an error message is emitted.

```
setup():
  Serial.begin(9600)
  dht.begin()

loop():
  read humidity + temperature + light
  if DHT data invalid: print diagnostic + light
  else: print temperature,humidity,light
  delay(2000)
```

3.2 Peripheral Configuration and Timing

Function	Configuration	Engineering interpretation
UART	<code>`Serial.begin(9600)`</code>	Used for telemetry output
DHT sensor	DHT11 on D2	Library-driven digital acquisition
ADC	<code>`analogRead(A0)`</code>	Reads LDR divider output
Sampling interval	<code>`delay(2000)`</code>	Nominal 2-second polling interval
Timer interrupts	None shown	No explicit ISR-based scheduler is implemented
Register-level configuration	None shown	Arduino HAL/API abstractions are used

3.3 Telemetry Data Formatting

Normal packet:
<temperature>,<humidity>,<light_intensity>

Example:
38.00,85.00,975

Fault packet:
DHT Sensor Error! - Light reading: <light_intensity>

The normal packet is CSV-like rather than JSON or a binary frame. This is easy to inspect in Proteus and easy to parse with a Python serial reader. The trade-off is that there is no explicit timestamp, device identifier, sensor-status field, checksum, or framing marker in the firmware output. The logger therefore supplies part of the temporal context and must infer or assign status from the received line.

3.4 Firmware Engineering Assessment

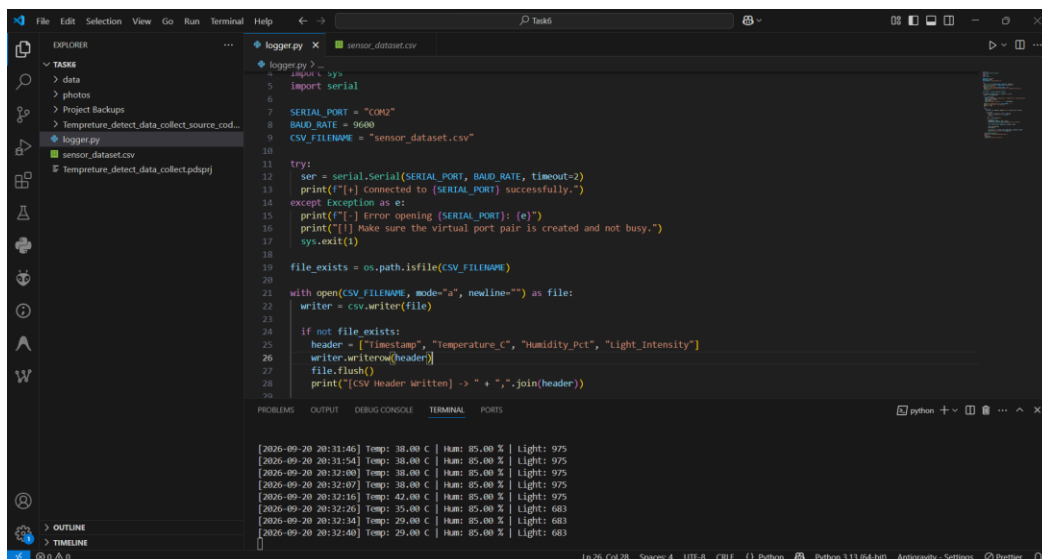
Observed interface gap

The ML inference function expects fields such as rainfall_mm, wind_speed_kmh, cloud_cover_percent, weather_condition and sensor_status, but the embedded firmware only transmits temperature, humidity and light intensity. A production system should introduce a versioned telemetry schema or an adapter that explicitly declares which fields are sensor-derived and which are externally enriched.

4. Simulation Interface & Telemetry Logger (Python Middleware)

4.1 Proteus Virtual COM Interface

The supplied logger screenshot shows a Python script configured for `COM2` at 9600 baud. The script imports `serial`, attempts to open the configured port with a 2-second timeout, reports successful connection, and terminates with an error message if the port cannot be opened. The logger writes into `sensor\_dataset.csv` in append mode and creates a header row when the file does not yet exist.



```

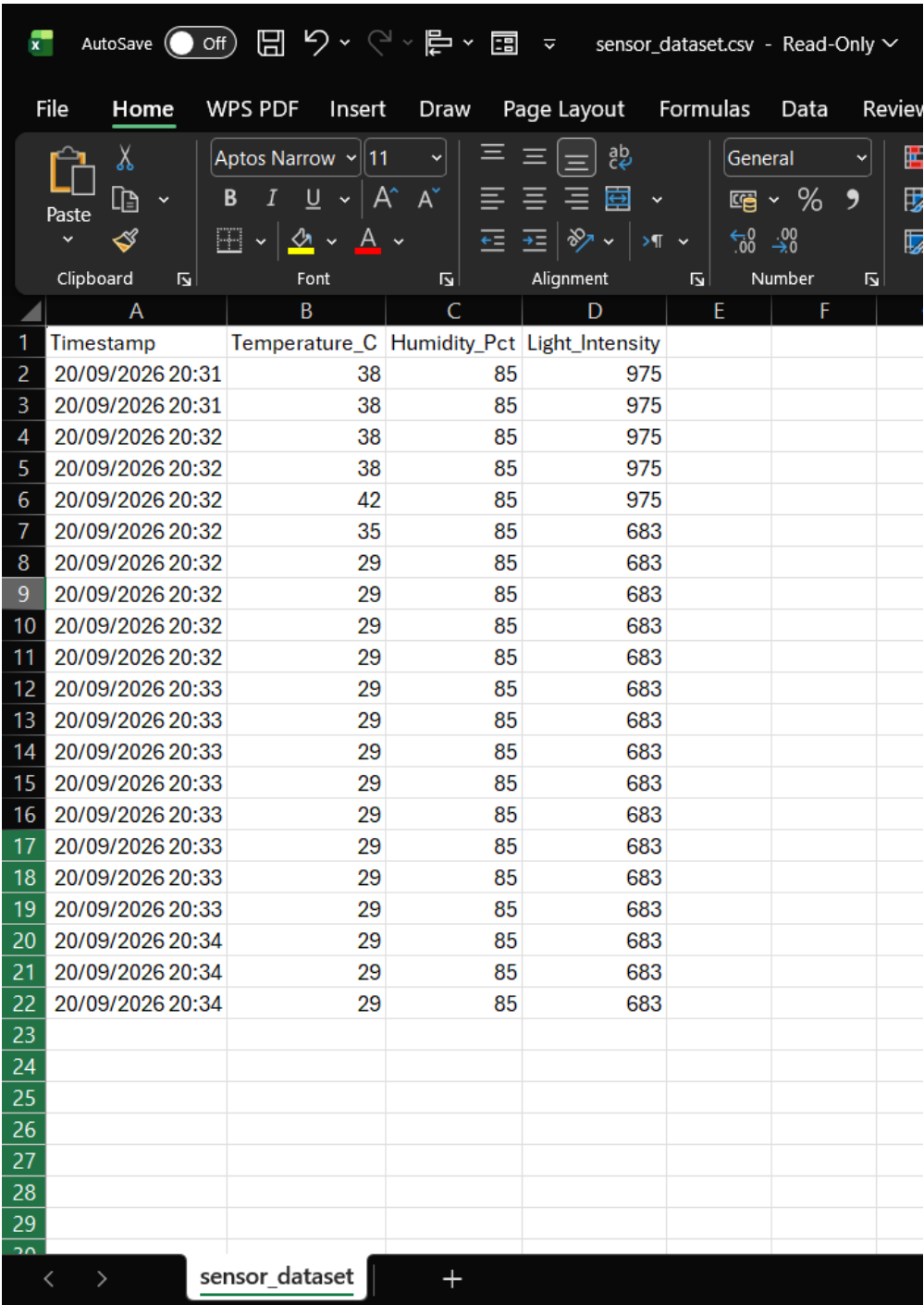
1  logger.py
2  import sys
3  import serial
4
5  SERIAL_PORT = "COM2"
6  BAUD_RATE = 9600
7  CSV_FILENAME = "sensor_dataset.csv"
8
9  try:
10     ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
11     print(f"[+] connected to {SERIAL_PORT} successfully.")
12 except Exception as e:
13     print(f"[!] Error opening {SERIAL_PORT}: {e}")
14     print("[!] Make sure the virtual port pair is created and not busy.")
15     sys.exit(1)
16
17 file_exists = os.path.isfile(CSV_FILENAME)
18
19 with open(CSV_FILENAME, mode="a", newline="") as file:
20     writer = csv.writer(file)
21
22 if not file_exists:
23     header = ["timestamp", "temperature_c", "humidity_pct", "light_intensity"]
24     writer.writerow(header)
25     file.flush()
26     print("[CSV Header Written] -> ", ", ".join(header))
27
28 [2026-09-20 20:31:46] Temp: 38.00 C | Hum: 85.00 % | Light: 975
29 [2026-09-20 20:31:54] Temp: 38.00 C | Hum: 85.00 % | Light: 975
30 [2026-09-20 20:32:00] Temp: 38.00 C | Hum: 85.00 % | Light: 975
31 [2026-09-20 20:32:07] Temp: 38.00 C | Hum: 85.00 % | Light: 975
32 [2026-09-20 20:32:16] Temp: 42.00 C | Hum: 85.00 % | Light: 975
33 [2026-09-20 20:32:20] Temp: 35.00 C | Hum: 85.00 % | Light: 683
34 [2026-09-20 20:32:34] Temp: 29.00 C | Hum: 85.00 % | Light: 683
35 [2026-09-20 20:32:40] Temp: 29.00 C | Hum: 85.00 % | Light: 683

```

Figure 2. Supplied Python logger view showing COM2, 9600 baud and CSV initialization.

4.2 Storage Format and Captured Data

The captured CSV sample contains four fields: Timestamp, Temperature_C, Humidity_Pct and Light_Intensity. Multiple records may be collected within the same minute, which is consistent with the 2-second embedded sampling period and demonstrates that the hardware log is a high-frequency telemetry stream rather than the hourly ML training representation.



	A	B	C	D	E	F
1	Timestamp	Temperature_C	Humidity_Pct	Light_Intensity		
2	20/09/2026 20:31	38	85	975		
3	20/09/2026 20:31	38	85	975		
4	20/09/2026 20:32	38	85	975		
5	20/09/2026 20:32	38	85	975		
6	20/09/2026 20:32	42	85	975		
7	20/09/2026 20:32	35	85	683		
8	20/09/2026 20:32	29	85	683		
9	20/09/2026 20:32	29	85	683		
10	20/09/2026 20:32	29	85	683		
11	20/09/2026 20:32	29	85	683		
12	20/09/2026 20:33	29	85	683		
13	20/09/2026 20:33	29	85	683		
14	20/09/2026 20:33	29	85	683		
15	20/09/2026 20:33	29	85	683		
16	20/09/2026 20:33	29	85	683		
17	20/09/2026 20:33	29	85	683		
18	20/09/2026 20:33	29	85	683		
19	20/09/2026 20:33	29	85	683		
20	20/09/2026 20:34	29	85	683		
21	20/09/2026 20:34	29	85	683		
22	20/09/2026 20:34	29	85	683		
23						
24						
25						
26						
27						
28						
29						
30						

Figure 3. Supplied sensor_dataset.csv sample captured from the Proteus/serial path.

4.3 Buffering, Fault Handling and State

The runtime inference layer contains explicit last-valid state for temperature, humidity, rainfall, wind, and cloud cover. If `sensor\_status` is not `Normal`, the implementation reuses the most recent valid values. This is a forward-fill strategy at the feature-ingestion boundary.

```
system_memory = {
    last_valid_temp,
    last_valid_humidity,
    last_valid_rain,
    last_valid_wind,
    last_valid_cloud,
    temp_lag_1
}

if sensor_status != 'Normal':
    use last valid weather values
else:
    use incoming sensor values
    update last-valid memory
```

The variable `temp\_lag\_1` is maintained as a one-sample temporal state and updated after each processed record. It is not a circular buffer in the strict sense; it stores only one previous temperature. More importantly, it is not part of the trained 18-feature matrix and is therefore removed when the live row is reindexed to the model's expected feature list. It currently has no predictive effect on the serialized model.

4.4 Error-Recovery Characteristics

Condition	Current implementation	Recommended control
COM port open failure	Exception message + `sys.exit(1)`	Reconnect with backoff; log the root cause
DHT invalid reading	Firmware emits diagnostic text	Encode structured status in telemetry protocol
Non-normal runtime status	Reuse last valid values	Add stale-data age and maximum hold time
Missing model feature	Inference fills with zero	Fail-fast for required fields to avoid silent schema drift
Unknown weather condition	No dummy column is activated	Validate against training categories instead of implicitly behaving like the reference category

5. Machine Learning Pipeline & Mathematics

5.1 Historical Dataset Structure

The notebook reports an input dataset of 526,080 records and 14 columns, with zero duplicate rows and no missing values in the profiled snapshot. The project brief describes coverage across 20 districts; the notebook itself confirms the district identifier and district-name fields and then isolates Downtown for model development.

Field	Notebook type	Modeling role
timestamp	object -> datetime	Temporal anchor
district_id	object	Spatial identifier; dropped from final X
temperature_c	float64	Current temperature predictor
feels_like_c	float64	Excluded due to multicollinearity with temperature
humidity_percent	float64	Current humidity predictor
rainfall_mm	float64	Used to derive rainfall_log / flags
wind_speed_kmh	float64	Used to derive wind_speed_log / flags
cloud_cover_percent	float64	Current cloud-cover predictor
weather_condition	object	One-hot encoded
heatwave_flag	int64	Event flag
heavy_rain_flag	int64	Event flag
storm_flag	int64	Event flag
sensor_status	object	Operational status; excluded from final X
district_name	object	Used to isolate Downtown; later removed from X

5.2 Data Cleaning and Group-Based Temporal Handling

The notebook converts the timestamp into a Pandas datetime representation, validates bounded percentage variables, filters the data to `district\_name == 'Downtown'`, and evaluates outliers. An IQR capping function is defined, but the notebook's narrative explicitly concludes that IQR treatment is not appropriate for the strongly skewed rainfall and wind-speed variables, so logarithmic transformation is used instead.

Interpolation status

A group-wise temporal interpolation strategy is appropriate for real sensor dropouts, but no executed interpolation operation appears in the supplied notebook cells. The profiled source snapshot contains no missing values, so interpolation is not required for the current training run. For production, any interpolation must be constrained by district and bounded by a maximum gap duration.

5.3 Cyclical Time Encoding

Hour-of-day and month-of-year are periodic variables. Encoding them as raw integers would impose an artificial discontinuity between 23:00 and 00:00 or between December and January. The notebook therefore projects the phase onto the unit circle.

$$\begin{aligned}\text{hour_sin} &= \sin(2\pi \cdot \text{hour} / 24), & \text{hour_cos} &= \cos(2\pi \cdot \text{hour} / 24) \\ \text{month_sin} &= \sin(2\pi \cdot \text{month} / 12), & \text{month_cos} &= \cos(2\pi \cdot \text{month} / 12)\end{aligned}$$

The paired sine/cosine representation preserves both phase and periodic adjacency. In a linear model, this allows the coefficient matrix to represent smooth daily and annual effects without treating the endpoints as far apart.

5.4 Logarithmic Transformations

Rainfall and wind speed are transformed with `log1p`, equivalent to $\ln(1+x)$. The addition of one preserves zero-valued measurements while compressing the upper tail:

$$x' = \ln(1 + x)$$

The purpose is not to eliminate physically meaningful extremes, but to reduce skew so that a linear estimator is less dominated by very large observations.

5.5 Categorical One-Hot Representation

The `weather\_condition` field is encoded with `pd.get\_dummies(..., drop\_first=True)`. The resulting six binary columns in the final matrix are Cloudy, Heatwave, Heavy Rain, Partly Cloudy, Rain and Storm. The omitted category is the reference level. This design avoids redundant columns under ordinary linear-model assumptions.

5.6 Multi-Step Horizon Shift

For each valid forecast origin t , six direct future targets are created by shifting the temperature series backward relative to the feature row:

$$\begin{aligned}y_t &= [T_{\{t+1\}}, T_{\{t+2\}}, T_{\{t+3\}}, T_{\{t+4\}}, T_{\{t+5\}}, T_{\{t+6\}}] \\ \text{target_t_h} &= \text{temperature_c.shift}(-h), \quad h = 1, \dots, 6\end{aligned}$$

The negative shift means the value observed h records ahead is aligned with the current feature vector. The final six rows cannot be assigned a complete target vector and are consequently removed before model fitting. This is a direct multi-horizon formulation; the model does not recursively feed its own forecast back into the next forecast.

5.7 Final Feature Space

Feature	Meaning	Transformation / type
temperature_c	Current measured temperature	Direct
humidity_percent	Relative humidity	Direct
cloud_cover_percent	Cloud cover	Direct / enriched
heatwave_flag	Heatwave event indicator	Binary
heavy_rain_flag	Heavy rainfall event indicator	Binary
storm_flag	Storm / high-wind indicator	Binary
rainfall_log	Rainfall transformed with $\ln(1+x)$	Continuous
wind_speed_log	Wind transformed with $\ln(1+x)$	Continuous
hour_sin	Daily cycle phase	Continuous
hour_cos	Daily cycle phase	Continuous
month_sin	Annual cycle phase	Continuous
month_cos	Annual cycle phase	Continuous
weather_condition_Cloudy	One-hot condition	Binary
weather_condition_Heatwave	One-hot condition	Binary
weather_condition_Heavy Rain	One-hot condition	Binary
weather_condition_Partly Cloudy	One-hot condition	Binary
weather_condition_Rain	One-hot condition	Binary
weather_condition_Storm	One-hot condition	Binary

The notebook confirms the final matrices at 26,298 Downtown rows, 18 predictor columns and 6 target columns. The six target columns are `target\_t\_1` through `target\_t\_6`, and the serialized artifact stores the exact 18-column predictor order.

6. Model Training & Evaluation Breakdown

6.1 Training Protocol

The notebook uses an 80/20 chronological holdout. The first 80% of rows form the training partition and the final 20% form the test partition. No random shuffling is applied. This preserves temporal ordering and prevents the test period from influencing model fitting.

6.2 Multi-Output Linear Regression

A single `LinearRegression` estimator is fitted against all six target columns. The mathematical form is:

$$\hat{Y} = XW + b$$

where X is the 18-feature input matrix, W is the learned 18×6 coefficient matrix, and b contains six intercepts. Each row therefore produces six direct temperature forecasts without recursive rollout.

6.3 Evaluation Metrics

Performance is evaluated independently for each horizon using mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2).

$$\begin{aligned} \text{MAE} &= (1/n) \sum |y_i - \hat{y}_i| \\ \text{RMSE} &= \sqrt{(1/n) \sum (y_i - \hat{y}_i)^2} \\ R^2 &= 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2] \end{aligned}$$

Forecast Horizon	MAE (°C)	RMSE (°C)	R ² Score
t+1 (Hour 1)	1.75	2.22	0.9445
t+2 (Hour 2)	1.78	2.26	0.9429
t+3 (Hour 3)	1.81	2.29	0.9411
t+4 (Hour 4)	1.82	2.28	0.9415
t+5 (Hour 5)	1.80	2.29	0.9413
t+6 (Hour 6)	1.79	2.27	0.9421
Mean	1.79	2.27	0.9422

6.4 Behavioral Analysis of Forecast Stability

The six horizons show a narrow error band: MAE ranges from 1.75 to 1.82 °C, RMSE from 2.22 to 2.29 °C, and R² from 0.9411 to 0.9445. This indicates that forecast quality does not collapse as the horizon advances from one to six hours.

- **Direct multi-output formulation.** Each horizon has its own coefficient vector but shares the same current-state inputs, so forecast errors do not accumulate through recursive feedback.
- **Cyclical time vectors.** Hour and month sine/cosine terms provide structured periodic information that remains useful for all six horizons.
- **Regression-to-the-mean tendency.** With a linear estimator and a finite set of predictors, predictions for unusual states can be pulled toward the conditional mean. This is a plausible mechanism for stable aggregate error but can reduce fidelity at extreme temperatures.
- **Feature-space limitation.** The model relies on contemporaneous exogenous variables and does not include a trained lagged-temperature feature. Therefore, temporal persistence is represented indirectly through the current weather state and time encodings rather than through autoregressive recurrence.

Interpretation boundary

The supplied metrics establish strong goodness-of-fit on the chronological test partition for the documented dataset. They do not, by themselves, establish field accuracy on the Proteus sensor stream or on unseen weather regimes. Hardware validation and an independently time-indexed field test are required for that claim.

7. Real-Time Hardware Inference & Production Deployment

7.1 Live Payload to Model Feature Transformation

The supplied `inference.py` demonstrates how a structured live payload is converted into the model's exact feature order. It loads the serialized artifact, reads the `features` list stored next to the model, builds cyclical time features, applies `log1p` to rainfall and wind, creates event flags, one-hot encodes the weather condition, inserts default values for missing schema columns, and finally reorders the row to match the training contract.

```
raw payload
|
+-- sensor_status != Normal ? last-valid fallback : live values
|
+-- timestamp -> hour_sin/hour_cos/month_sin/month_cos
+-- rainfall -> rainfall_log
+-- wind      -> wind_speed_log
+-- weather_condition -> one-hot columns
+-- flags -> provided values or derived thresholds
|
v
18-column DataFrame in stored feature order
|
v
model.predict(X_live)
|
v
6 direct temperature forecasts
```

7.2 Runtime Feature Contract

Input / state	Model role	Processing	Current sensor coverage
temperature_c	Yes	Direct	DHT11
humidity_percent	Yes	Direct	DHT11
cloud_cover_percent	Yes	Direct	Not present in schematic
heatwave_flag	Yes	Direct / derive if absent	Derived in inference; no dedicated hardware
heavy_rain_flag	Yes	Direct / derive if absent	Requires rainfall source
storm_flag	Yes	Direct / derive if absent	Requires wind source
rainfall_mm	Indirect	`log1p`	Requires rainfall source
wind_speed_kmh	Indirect	`log1p`	Requires wind source
timestamp	Yes	Cyclical encoding	Logger / edge clock
weather_condition	Yes	One-hot mapping	Requires enrichment
district_id	No	Ignored after feature construction	Payload includes D01
feels_like_c	No	Ignored	Payload includes field
light_intensity	No	Ignored by ML model	LDR supplies it
sensor_status	No	Controls fallback logic	Not directly emitted by firmware

7.3 Critical Integration Mismatches

Interface gap	Impact	Engineering action
2-second sensor telemetry vs hourly model cadence	Feature row represents a different temporal resolution from the training data	Aggregate / resample live telemetry into hourly statistics or retrain at the acquisition cadence
Hardware outputs only T/RH/light	Six of the 18 model dimensions depend on rain, wind, cloud or categorical weather state	Add sensors or integrate a trusted upstream weather source
Firmware error is text, inference expects sensor_status	Failure propagation is ambiguous	Define an explicit status code / packet field
Light intensity is logged but absent from training	A collected sensor cannot improve the current model without retraining	Either retain it as telemetry-only or retrain with light features
temp_lag_1 is maintained but not trained	Runtime state adds complexity without changing prediction	Remove it or retrain with an actual lag feature
Missing feature columns are silently zero-filled	Schema errors can be masked	Distinguish optional from required fields and fail fast on required omissions
Unknown weather condition maps to all-zero dummy vector	This can become indistinguishable from the reference category	Validate categorical vocabulary and reject unknown classes

7.4 Deployment Options

Deployment pattern	Execution model	Advantages	Main constraints
Python + SBC gateway	Keep `.pkl` + Python feature adapter	Closest to current artifact; easy updates and diagnostics	Higher memory/runtime dependency; requires trusted Python environment
Bare-metal MCU inference	Compile W and b into C/C++	Deterministic, low runtime overhead, no Python dependency	Must reproduce all preprocessing; model artifact cannot be used directly; sensor coverage must match
Hybrid edge gateway	MCU senses; gateway aggregates + infers	Separates hard real-time sensing from flexible ML/ETL	Adds one system boundary and synchronization requirement

7.5 Direct MCU Mathematical Port

Because the estimator is linear, once coefficients are exported the inference operation is a matrix-vector product. For six outputs:

$$\hat{y}_j = b_j + \sum_{i=1}^{18} w_{(i,j)} x_i, \quad j = 1, \dots, 6$$

The arithmetic core therefore contains $18 \times 6 = 108$ multiply-accumulate terms per inference, plus six bias additions. There is no matrix inversion at runtime; least-squares fitting occurs during training. The computationally heavier part

on a small MCU is not the linear layer itself but reproducing the feature transformations, particularly logarithm and trigonometric functions.

- For strict deterministic embedded execution, precompute sine/cosine values with lookup tables or fixed-point approximations if floating-point math is expensive.
- Export the learned coefficients and intercepts in a fixed, versioned array format and record the feature order alongside the firmware build.
- Quantization is possible, but must be validated against the original floating-point model to bound forecast error.
- Do not port the estimator alone: the same one-hot vocabulary, flag thresholds and scaling conventions must be preserved.

8. Verification, Results & Failure Modes

8.1 Verification Strategy

Scenario	Expected behavior	Verification point
Day -> night transition	No discontinuity caused by hour=23 to hour=0	Check hour_sin/hour_cos continuity and forecast smoothness
Summer heatwave	Heatwave flag activates at configured threshold when not supplied	Verify `temp > 40` default behavior and forecast response
Heavy rainfall	Heavy-rain flag activates when rainfall exceeds threshold	Verify `rain > 15` and `log1p(rain)` consistency
High wind / storm	Storm flag activates when wind exceeds threshold	Verify `wind > 40` threshold and transformed wind value
Winter freeze	Low temperatures pass without clipping artifacts	Check model residuals at low-temperature test cases
DHT timeout	Runtime continues using last-valid weather state when status is non-normal	Verify fallback age and alarm behavior
Unknown weather category	System rejects or explicitly maps unsupported categories	Prevent silent all-zero one-hot encoding
Serial disconnection	Logger retries and records the outage	Add backoff/reconnect test
Cold start	Initial-state values do not dominate the first forecast unexpectedly	Validate initialization policy for last-valid state
Hourly aggregation boundary	Records crossing an hour do not leak future data	Validate window close time and timestamp alignment

8.2 Failure Mode and Effects Analysis

Failure mode	Effect	Production severity	Mitigation
DHT read failure	Text diagnostic replaces structured payload	High	Versioned structured packet with status + reason code
Serial port unavailable	Logger exits	High	Retry with backoff + watchdog
Stale fallback values	Forecast can become biased during prolonged outage	High	Track stale age; alert after maximum hold time
Missing upstream rain/wind/cloud	18-feature row cannot be physically reconstructed	High	Additional sensors or validated external feed
2 s vs 1 h cadence mismatch	Model receives temporally inconsistent input	High	Hourly aggregator with explicit windowing
Unknown categorical value	Can behave like reference dummy vector	Medium	Vocabulary validation + explicit UNKNOWN category
Model/feature order drift	Incorrect prediction with no obvious runtime error	High	Bundle schema, version and validation checks
Pickle artifact tampering / incompatibility	Load failure or unsafe deserialization	High	Trusted artifact provenance, pinned runtime versions, secure deployment
Extremes outside training distribution	Potential extrapolation error from linear model	Medium/High	OOD monitoring and periodic retraining

8.3 Demonstrated Runtime Inference Example

The supplied inference demonstration uses a live payload timestamped `2026-07-15 21:00:00` for D01 with temperature 28 °C, humidity 55%, zero rainfall, wind speed 7.5 km/h, zero cloud cover, `Clear` weather condition, and Normal sensor status. The model is then invoked once to produce a six-element forecast vector.

```
Input:
timestamp = 2026-07-15 21:00:00
district_id = D01
temperature_c = 28
humidity_percent = 55
rainfall_mm = 0
wind_speed_kmh = 7.5
cloud_cover_percent = 0
weather_condition = Clear
sensor_status = Normal

Demonstration forecast from notebook artifact:
[25.44, 24.44, 23.22, 21.96, 20.64, 19.47] °C
```

This vector demonstrates that the serialized model can be reloaded and called in a live-style path. It should be interpreted as a software integration test, not as independent evidence of field forecast accuracy.

9. Engineering Conclusions & Recommended Integration Path

The supplied artifacts form a coherent proof-of-concept chain from embedded sensing to model inference: the Proteus circuit acquires physical variables, the Arduino firmware serializes telemetry, Python captures the stream, the ML notebook learns a direct six-horizon regression model, and `inference.py` reconstructs the trained feature schema for real-time prediction.

The principal engineering issue is not the linear model itself; it is interface consistency between the embedded sensing plane and the 18-feature hourly model contract. In particular, the current hardware does not measure all variables required by the model, the acquisition rate is 2 seconds while the learning problem is hourly, and the firmware failure message is not a structured status field.

9.1 Recommended Production Integration Path

1. Standardize a versioned telemetry envelope containing timestamp, device ID, sensor status, temperature, humidity, light intensity, and optional environmental enrichment fields.
2. Add an edge aggregation service that converts the 2-second telemetry stream into the same hourly semantic representation used by the training dataset.
3. Define a source of rainfall, wind speed and cloud cover: physical sensors, validated local gateway inputs, or a trusted upstream weather feed. Do not silently fabricate these fields.
4. Encode weather-condition categories against the exact training vocabulary and add explicit validation for unknown categories.
5. Decide whether light intensity should remain telemetry-only or be incorporated into a retrained feature space.
6. Either remove `temp\_lag\_1` from the runtime state or retrain the model with an actual lagged-temperature feature; the current artifact does not use it.
7. Bundle the estimator, feature schema, category vocabulary, threshold definitions and model version as one deployment contract.
8. For edge deployment, prefer a trusted Python/SBC path first for exact parity. A direct MCU port should follow only after the feature pipeline is frozen and validated against a golden test vector.
9. Establish field validation using time-separated data and explicit extreme-weather test sets before claiming operational forecast performance.

9.2 Acceptance Criteria for a Production-Ready Release

Criterion	Pass condition
Sensor protocol	All normal and fault packets are machine-readable and versioned
Temporal semantics	Live inference uses the same hourly semantics as training
Feature completeness	All 18 features are deterministically populated or explicitly rejected
Categorical consistency	All weather-condition classes are validated against the training vocabulary
Fallback control	Stale-data fallback has a time limit and alarm
Model integrity	Serialized model, feature order and version are cryptographically traceable / provenance-controlled
Golden-vector test	Edge implementation matches the reference Python model within a defined numeric tolerance
Forecast accuracy	Field test metrics are reported separately from the historical holdout metrics

Appendix A. End-to-End Interface Contract

The following table consolidates the fields visible across the embedded logger, the historical training dataset and the inference demonstration. It is intended as an integration contract rather than a claim that all fields currently originate from the same physical sensor set.

Field	Proteus / firmware	CSV logger	Training	Inference	Role
timestamp	Not emitted by firmware	Yes	Yes	Yes	Temporal alignment
temperature_c	Yes	Yes	Yes	Yes	Core predictor
humidity_percent	Yes	Yes	Yes	Yes	Core predictor
light_intensity	Yes	Yes	No	No	Telemetry only in current ML
rainfall_mm	No	No	Yes	Yes	Derived / external enrichment
wind_speed_kmh	No	No	Yes	Yes	Derived / external enrichment
cloud_cover_percent	No	No	Yes	Yes	Derived / external enrichment
weather_condition	No	No	Yes	Yes	One-hot categorical
heatwave_flag	No	No	Yes	Yes	Binary event feature
heavy_rain_flag	No	No	Yes	Yes	Binary event feature
storm_flag	No	No	Yes	Yes	Binary event feature
sensor_status	No (implicit error text)	Not shown in header	Yes	Yes	Fault/fallback control
district_id	No explicit field	No	Yes	Yes	Scope identifier, not a final feature

Appendix B. Model Feature Dictionary

The model artifact reports 18 expected features. Their exact order is serialized with the LinearRegression estimator and is used by the live inference adapter. The column order is therefore part of the model interface, not merely a convenience.

#	Feature	Type	Construction	Typical range / unit
1	temperature_c	float	Direct	°C
2	humidity_percent	float	Direct	%
3	cloud_cover_percent	float	Direct	%
4	heatwave_flag	int	Binary	0/1
5	heavy_rain_flag	int	Binary	0/1
6	storm_flag	int	Binary	0/1
7	rainfall_log	float	ln(1+rainfall_mm)	dimensionless
8	wind_speed_log	float	ln(1+wind_speed_kmh)	dimensionless
9	hour_sin	float	sin(2πth/24)	[-1,1]
10	hour_cos	float	cos(2πth/24)	[-1,1]
11	month_sin	float	sin(2πtm/12)	[-1,1]
12	month_cos	float	cos(2πtm/12)	[-1,1]
13	weather_condition_Cloudy	int	One-hot	0/1
14	weather_condition_Heatwave	int	One-hot	0/1
15	weather_condition_Heavy Rain	int	One-hot	0/1
16	weather_condition_Partly Cloudy	int	One-hot	0/1

#	Feature	Type	Construction	Typical range / unit
17	weather_condition_Rain	int	One-hot	0/1
18	weather_condition_Storm	int	One-hot	0/1

Appendix C. Artifact Traceability & Technical References

C.1 Supplied Artifacts

Artifact	Supplied file	Primary evidence used
Proteus schematic	protues_simulation.png	Hardware topology, DHT11, LDR, UART/Virtual Terminal
Embedded firmware	Tempreture_detect_data_collect_source_code.ino.ino	DHT11 + LDR acquisition and UART formatting
Python logger screenshot	python_script.png	COM2, 9600 baud, CSV initialization
Sensor CSV screenshot	CSV_Data_file.png	Captured timestamp / temperature / humidity / light structure
ML notebook	ML_Model_documented.ipynb	Data preparation, feature engineering, target construction, training, evaluation, export
Inference implementation	inference.py	Live feature adapter, fallback state, schema ordering and prediction
Evaluation screenshot	Model Evaluation.png	Final MAE / RMSE / R ² values per horizon

C.2 Technical References

- [scikit-learn LinearRegression](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html) — https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- [pandas DataFrame.shift](https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.shift.html) — <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.shift.html>
- [joblib persistence](https://joblib.readthedocs.io/en/latest/persistence.html) — <https://joblib.readthedocs.io/en/latest/persistence.html>
- [Arduino Serial reference](https://docs.arduino.cc/language-reference/en/functions/communication/serial/) — <https://docs.arduino.cc/language-reference/en/functions/communication/serial/>

Final engineering position

The project is technically credible as a proof of concept and demonstrates a complete path from sensing to forecasting. Productionization should focus first on closing the data-contract boundary: temporal aggregation, sensor coverage, structured fault signaling, feature-vocabulary validation and artifact/schema versioning. The 18-feature linear estimator itself is computationally well suited to edge inference once those interfaces are deterministic.